Advanced Machine Learning

Project 1 Report

Wojciech Grunwald, Piotr Jurczyk and Krzysztof Krawiec
Warsaw University of Technology

April 2026

Abstract

The aim of this project is to analyze the performance of a logistic regression model when trained on datasets with missing labels. We investigate how different missing data mechanisms (MCAR, MAR, MNAR) affect the learning process and evaluate simple imputation strategies to maintain the model's accuracy.

Contents

1 Introduction **2**

2 FISTA Algorithm **2**
2.1 Overview and Motivation . . . 2
2.2 Implementation Details . . . 2

3 UnlabeledLogReg Implementation **3**
3.1 Prediction, Classification and Model Evaluation . . . 4

4 Experimental Results and Analysis **4**
4.1 Scope of the Experiments . . . 5
4.1.1 Evaluated Models . . . 5
4.2 General Performance (MCAR, MAR1, MAR2) . . . 5
4.3 The MNAR Anomaly: A Complete Model Collapse . . . 5
4.4 Sensitivity Analysis: Impact of the Missing Rate (MCAR) . . . 6

5 Code Availability **7**

6 References **7**

A Handling Missing Data **8**
A.1 MCAR . . . 8
A.2 MAR1 . . . 8
A.3 MAR2 . . . 8
A.4 MNAR . . . 8

B Preparing Datasets **9**

1 Introduction

In this project, we investigate the performance of logistic regression in the presence of missing labels. We consider different missing data mechanisms (described in details in **Appendix A**) and analyze their impact on model training and prediction.

To address this problem, we implement label-masking procedures, an optimization algorithm based on FISTA, and an extended logistic regression model capable of handling partially observed data. The proposed methods are evaluated on multiple datasets (described in details in **Appendix B**) with varying characteristics to assess their robustness and generalization ability.

2 FISTA Algorithm

2.1 Overview and Motivation

The **Fast Iterative Shrinkage-Thresholding Algorithm** (FISTA) is a first-order optimization method designed specifically for composite convex optimization problems. These problems typically involve minimizing an objective function that can be decomposed into two distinct components: a smooth, continuously differentiable loss function and a possibly non-smooth, convex penalty term:

$$\min_{\mathbf{w},b} F(\mathbf{w}, b) = f(\mathbf{w}, b) + g(\mathbf{w})$$

For standard binary classification, we assume the input labels are $y^{(i)} \in \{0, 1\}$. To formulate the logistic loss using margin-based notation, we map these to signed labels $\tilde{y}^{(i)} \in \{-1, 1\}$. Given feature vectors $\mathbf{x}^{(i)}$, the objective components are defined as:

$$f(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \log\left(1 + \exp\left(-y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b)\right)\right), \qquad g(\mathbf{w}) = \lambda\|\mathbf{w}\|_1$$

where $\lambda > 0$ acts as the regularization hyperparameter. To iteratively minimize the objective function formulated above, we employ the standard FISTA procedure outlined in Algorithm 1 [1].

Algorithm 1 FISTA with constant stepsize

Require: $L = L(f)$ ▷ A Lipschitz constant of ∇f
Require: $\mathbf{x}_0 \in \mathbb{R}^n$
Step 0: Initialize $\mathbf{y}_1 = \mathbf{x}_0$, $t_1 = 1$.
for $k = 1, 2, \ldots$ **do**
  $\mathbf{x}_k = p_L(\mathbf{y}_k)$
  $t_{k+1} = \frac{1+\sqrt{1+4t_k^2}}{2}$
  $\mathbf{y}_{k+1} = \mathbf{x}_k + \left(\frac{t_k - 1}{t_{k+1}}\right)(\mathbf{x}_k - \mathbf{x}_{k-1})$
end for

2.2 Implementation Details

In our programmatic implementation, the iterative optimization relies strictly on vectorized mathematical operations. To align with FISTA notation, we denote the model weights as $\mathbf{x}$ and the intercept as b. We initialize the weights $\mathbf{x}_0$ and intercept b_0 to zero. To facilitate acceleration, auxiliary momentum variables $\mathbf{y}_1 = \mathbf{x}_0$, a bias momentum $y_{1,b} = b_0$, and a time step $t_1 = 1$ are introduced. The implementation then strictly follows the algorithm. Let us focus on how $\mathbf{x}_k$ is calculated in each iteration, using the soft-thresholding operator:

$$\mathbf{x}_k = p_L(\mathbf{y}_k) = \mathcal{T}_{\alpha\lambda}(\mathbf{x}_{\text{temp}}) = \mathcal{T}_{\alpha\lambda}(\mathbf{x}_{\mathbf{y}_k} - \alpha\nabla_{\mathbf{y}_k} f))$$

where the proximal operator is defined as: $\mathcal{T}_\gamma(z) = \text{sign}(z) \cdot \max(|z| - \gamma, 0)$, which leads to the element-wise update:

$$\mathbf{x}_k = \text{sign}(\mathbf{x}_{\text{temp}}) \odot \max(|\mathbf{x}_{\text{temp}}| - \alpha\lambda, 0)$$

Because the intercept b remains unregularized, it is updated purely via the gradient step:

$$b_k = y_{k,b} - \alpha \nabla_{y_{k,b}} f.$$

Note, that to determine the sensitivity of the loss f_i with respect to the momentum weights $\mathbf{y}_k$, we first define the logit as $z_i = (\mathbf{x}^{(i)})^T \mathbf{y}_k + y_{k,b}$. Consequently: $\frac{\partial z_i}{\partial \mathbf{y}_k} = \mathbf{x}^{(i)}$. Applying the chain rule to the logistic loss function f_i with respect to the logit z_i yields:

$$\frac{\partial f_i}{\partial z_i} = \frac{1}{1 + \exp\left(-y^{(i)} z_i\right)} \cdot \exp\left(-y^{(i)} z_i\right) \cdot \left(-y^{(i)}\right).$$

which can be simplified to the residual error term δ_i:

$$\delta_i = \frac{\partial f_i}{\partial z_i} = \left(\sigma\left(y^{(i)} z_i\right) - 1\right) y^{(i)}$$

Using the chain rule, we obtain the gradient for a single observation as $\frac{\partial f_i}{\partial \mathbf{y}_k} = \delta_i \cdot \mathbf{x}^{(i)}$. To compute the total gradient across all n observations, we average the individual gradients. The iterative process continues until a predefined convergence criterion is satisfied, specifically when the Euclidean norm of the parameter update, $\|\mathbf{x}_k - \mathbf{x}_{k-1}\|_2$, falls beneath a strict tolerance threshold.

3 UnlabeledLogReg Implementation

To address the challenge of missing labels in the training data, we implemented a custom wrapper class named `UnlabeledLogReg`. This approach falls under the umbrella of semi-supervised learning and single-imputation techniques. The core idea is to leverage the fully observed portion of the data to estimate the missing labels before training the final predictive FISTA model.

The algorithm proceeds in the following steps:

1. **Data Partitioning:** The input training set $(\mathbf{X}, y_{obs})$ is split into two disjoint subsets based on the label values: the labeled set where $y_{obs} \neq -1$, and the unlabeled (missing) set where $y_{obs} = -1$.

2. **Imputer Training:** An imputation model is trained exclusively on the labeled subset. We implemented two distinct imputation strategies:

   - **k-Nearest Neighbors (k-NN):** Uses the 5 closest labeled samples in the feature space to vote on the missing label.
   - **Logistic Regression (Sklearn):** Fits a standard logistic regression model to the observed data to predict the missing labels.

3. **Label Imputation:** The trained imputer predicts the labels $\hat{y}$ for the unlabeled subset $\mathbf{X}_{unlabeled}$.

4. **Dataset Reconstruction:** A complete, pseudo-labeled target vector $y_{complete}$ is formed by concatenating the original y_{obs} (for the labeled part) and the predicted $\hat{y}$ (for the missing part).

5. **Final Model Training:** The custom `FistaLogisticRegression` model is then trained on the fully reconstructed dataset $(\mathbf{X}, y_{complete})$.

3.1 Prediction, Classification and Model Evaluation

Following optimization, the framework provides standard inference and evaluation methods:

- `predict_proba` method computes the posterior class probabilities
- `predict` method converts the continuous probability estimates into binary class labels
- `plot`: Visualizes the selected performance metric as a function of λ
- `plot_coefficients`): Illustrates the trajectory of individual feature coefficients as λ varies.

4 Experimental Results and Analysis

Note on reproducibility: The raw numerical data generated during all experiments, including comprehensive metrics for every dataset configuration and the full arrays from the sensitivity analysis, have been exported and are provided in the attached `results/` *directory as* `all_experiment_results.json` *and* `all_sensitivity_results.json`*.*

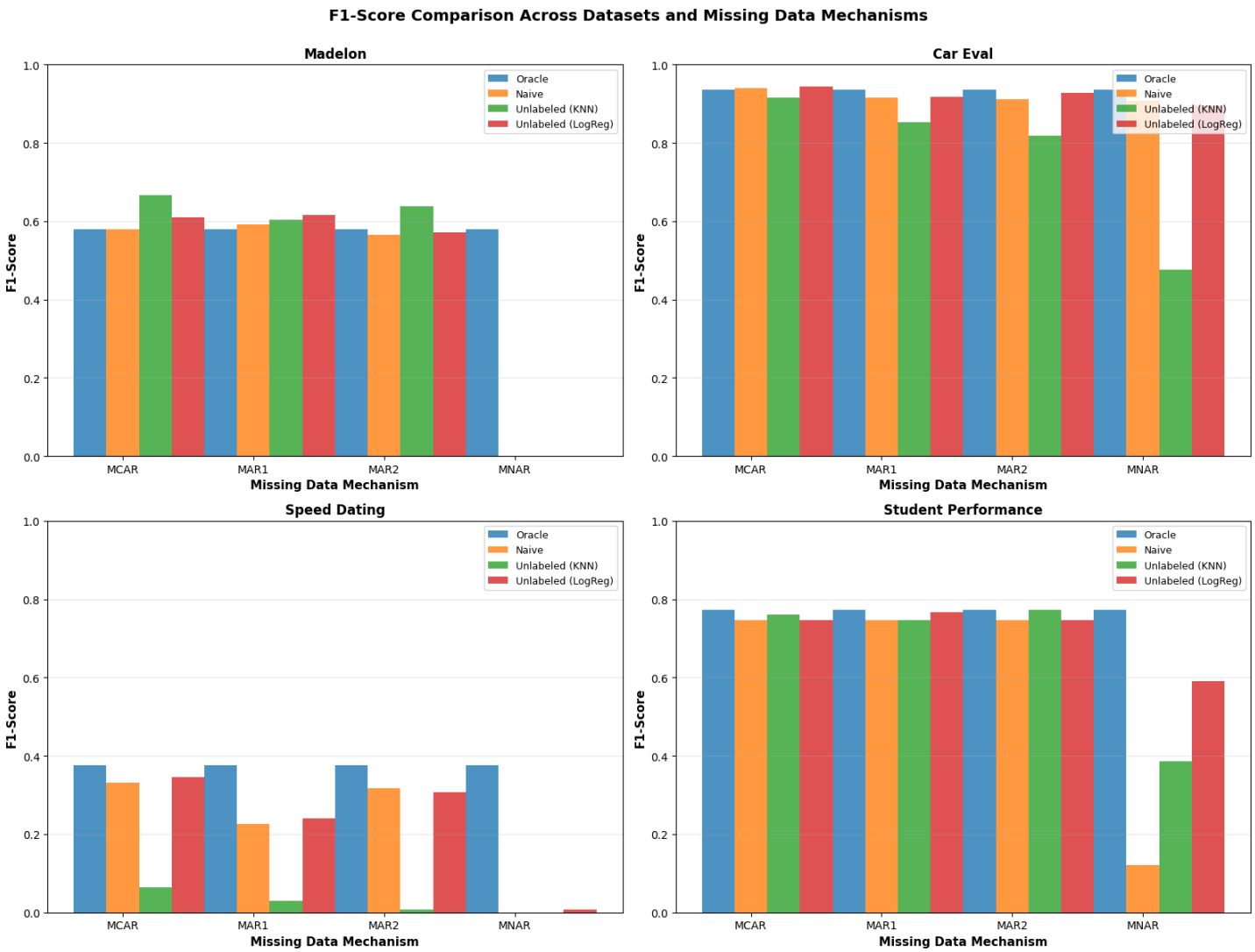


Figure 1: Master comparison of F1-Scores on the test set. Each subplot visualizes performance across four models (Oracle, Naive, Unlabeled KNN, Unlabeled LogReg) and four missing data mechanisms (MCAR, MAR1, MAR2, MNAR) for a specific dataset.

4.1 Scope of the Experiments

To thoroughly evaluate the robustness of our FISTA implementation and the `UnlabeledLogReg` imputation methods, we designed a comprehensive experimental setup.

4.1.1 Evaluated Models

For each dataset and each missing data mechanism (MCAR, MAR1, MAR2, MNAR), we evaluated four baselines:

1. **Oracle:** Trained on the original, uncorrupted training dataset (serves as the theoretical upper bound).
2. **Naive:** Trained using listwise deletion (ignoring all rows where $y = -1$).
3. **Unlabeled (KNN):** Imputed using k-NN before FISTA training.
4. **Unlabeled (LogReg):** Imputed using Sklearn's Logistic Regression before FISTA training.

4.2 General Performance (MCAR, MAR1, MAR2)

Performance was primarily evaluated using the F1-Score to account for class imbalances. Across the completely random (MCAR) and conditionally random (MAR1, MAR2) mechanisms, the performance of the models remained relatively stable compared to the Oracle baseline.

Surprisingly, the Naive approach (listwise deletion) proved to be highly competitive. In datasets like Car Eval and Student Performance, the Naive model often matched or even outperformed the imputation methods (KNN and LogReg). This happens because imputation models are not perfect and often predict incorrect labels for the missing data, introducing "noise" into the training set. The results suggest that the FISTA algorithm trains better on a smaller, but 100% ground-truth subset of data (Naive) rather than a larger dataset contaminated with incorrectly guessed pseudo-labels (Unlabeled methods), provided that the missing data is distributed randomly (MCAR/MAR).

4.3 The MNAR Anomaly: A Complete Model Collapse

The most striking observation from the experimental results is the catastrophic failure of all non-Oracle models under the **MNAR (Missing Not At Random)** mechanism. This is vividly illustrated in the performance tables and visualizations for the Madelon and Speed Dating datasets. For these configurations, the F1-Score for Naive, Unlabeled (KNN), and Unlabeled (LogReg) plummeted to exactly **0.0000**.

This behavior is a direct, mathematical consequence of the MNAR formulation used in the data generation process:

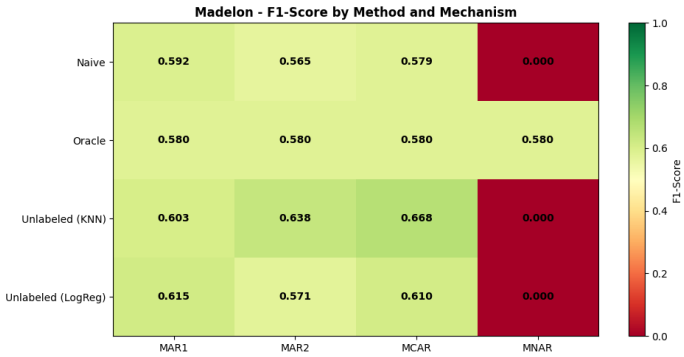
$$\text{combined_scores} = \text{scores} + (y_{true} \times 2 - 1)$$

Assuming binary labels $y \in \{0, 1\}$, this expression acts as a severe penalty. If the true class is $y = 1$, the score increases, exponentially raising the probability of the label being dropped. Consequently, the observable training set becomes drastically stripped of positive examples.

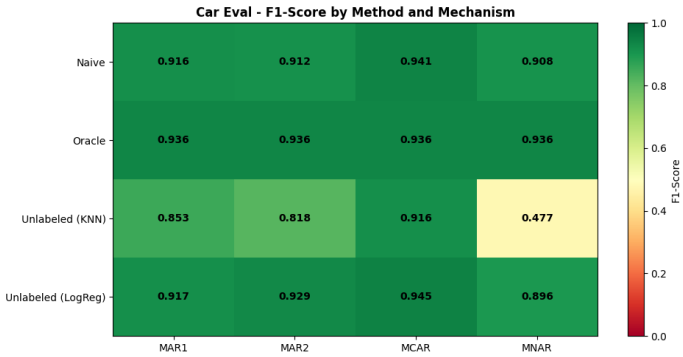
When the Naive model trains on this skewed data, it learns a strong prior towards the majority class ($y = 0$), pushing the intercept (b) to a large negative value. Similarly, the imputers in the Unlabeled methods are trained on a sea of zeros, causing them to predict 0 for almost all missing labels. Ultimately, the final FISTA model collapses into predicting a single class (Class 0) for the entire test set.

This model collapse is mathematically confirmed by observing the Balanced Accuracy metric for MNAR, which drops to exactly **0.5000**. In binary classification, a balanced accuracy of 0.5 perfectly indicates that the model has zero sensitivity (Recall = 0) and is predicting only one class. Because True Positives = 0, the F1-Score collapses to zero. Despite this, the ROC AUC often remains above 0.5 (e.g.,

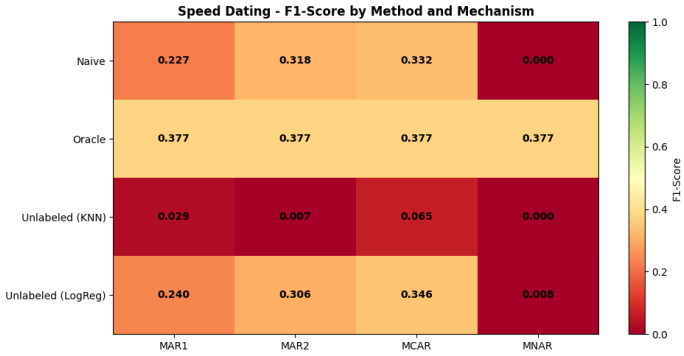
0.54 for Madelon MNAR), indicating the model retains some ranking capability, but fails completely at the hard threshold of 0.5.



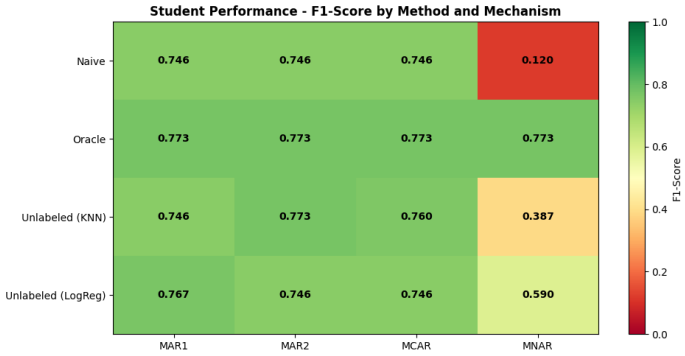
(a) Madelon F1-Scores



(b) Car Eval F1-Scores



(c) Speed Dating F1-Scores



(d) Student Performance F1-Scores

Figure 2: Numerical F1-Score heatmaps

4.4 Sensitivity Analysis: Impact of the Missing Rate (MCAR)

To understand the limits of our approaches, we evaluated the models' F1-Scores across an increasing probability of missing labels ($c \in [0.01, 0.90]$) under the MCAR assumption, as visualized in Figure 3.

- **Madelon:** The results become highly volatile. Madelon relies on complex, non-linear relationships without a single strong linear predictor. Removing labels drastically increases the variance of the FISTA gradients, leading to large oscillations in the F1-Score.
- **Car Eval:** The models demonstrate extreme robustness. Performance remains virtually identical to the Oracle up to a missing rate of $c = 0.60$. Beyond $c = 0.60$, the Unlabeled (KNN) method degrades sharply, while the Naive method and LogReg imputation remain relatively stable even with 80% of the labels missing.
- **Speed Dating:** We observe a steady, near-linear decline in the F1-Score as c increases. Due to the initial high imbalance (only 16.5% positive class), dropping data uniformly quickly removes the critical minority examples needed to establish a good decision boundary.
- **Student Performance:** Because the total dataset size is merely 395 samples (with only 237 in the training set), removing any data creates severe instability. The sensitivity plot exhibits massive variance, indicating that for very small datasets, the specific random seed (determining which exact rows are dropped) matters more than the imputation algorithm itself.

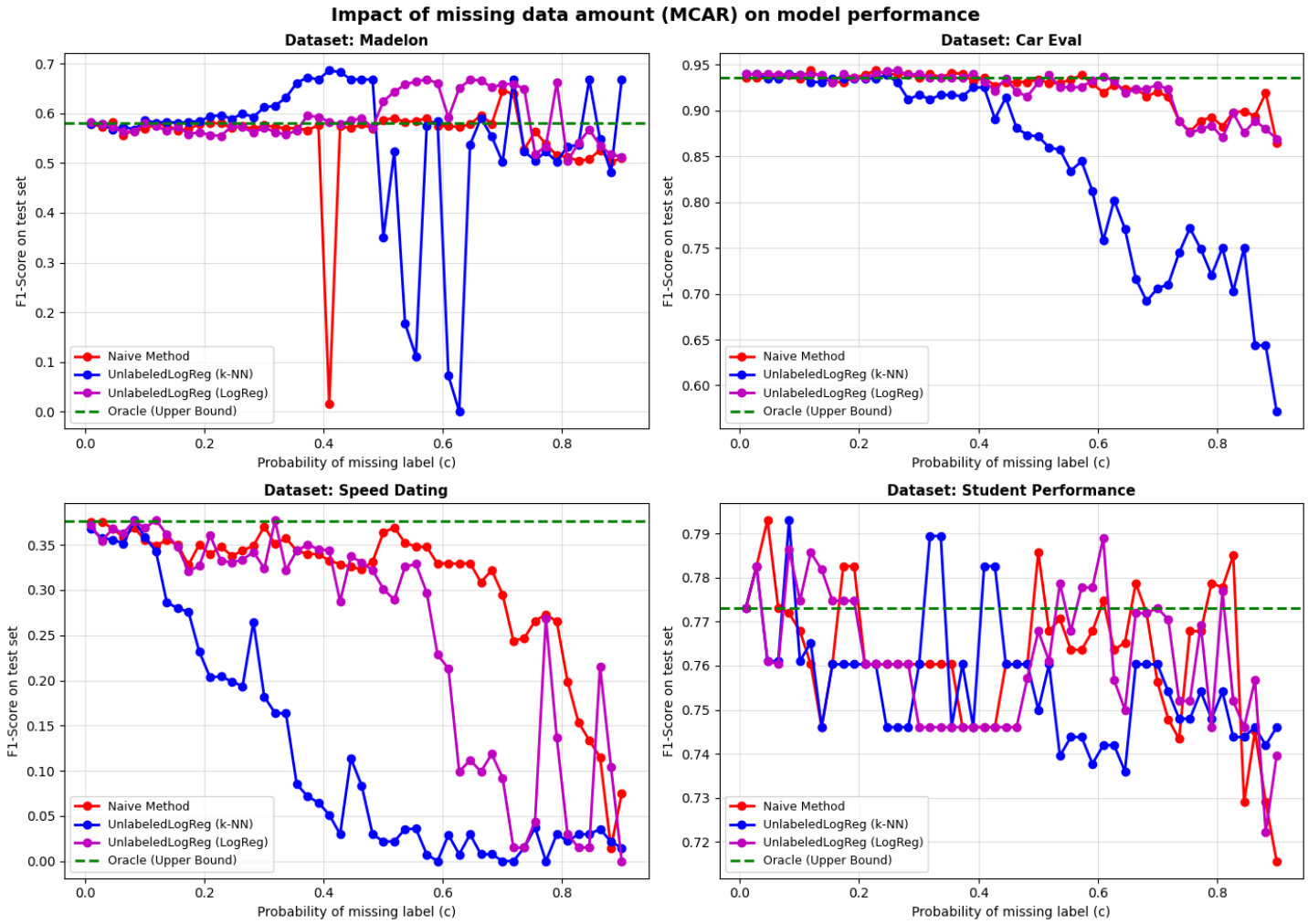


Figure 3: MCAR Sensitivity Dashboard. Plots illustrate the F1-Score on the test set as the probability of missing labels (c) increases from 0.01 to 0.90, comparing Naive, Unlabeled KNN, and Unlabeled LogReg methods against the green dashed Oracle baseline.

Overall, the sensitivity analysis proves that the Naive strategy is remarkably robust under MCAR conditions for sufficiently large and balanced datasets, whereas imputation methods (especially KNN) require careful tuning and sufficient data density to be effective.

5 Code Availability

The complete source code, including the implementation of the FISTA algorithm, UnlabeledLogReg class, and scripts used to generate the figures in this report, is available on GitHub at:
https://github.com/pitygiusz/AML-Project1

6 References

[1] Beck, A., & Teboulle, M. (2009). A fast iterative shrinkage-thresholding algorithm for linear inverse problems. SIAM journal on imaging sciences, 2(1), 183-202.

A Handling Missing Data

To evaluate the robustness of our algorithms, we synthetically injected missing values (denoted as $y = -1$) **exclusively into the training sets**, evaluating the performance on uncorrupted validation and test sets. We implemented four distinct data-missing mechanisms:

A.1 MCAR

In MCAR (Missing Completely At Random), missingness is independent of both X and Y:

$$P(S = 1 \mid X, Y) = c,$$

where $c \in [0, 1]$. Missing values are generated by sampling a binary indicator vector S from a Bernoulli distribution with parameter c. The observed target y_{obs} is obtained by replacing entries corresponding to $S = 1$ with a placeholder value (e.g., -1).

A.2 MAR1

In MAR1 (Missing At Random), missingness depends on a single feature X_j, where $j \sim \mathcal{U}(0, p-1)$:

$$P(S = 1 \mid X, Y) = P(S = 1 \mid X_j).$$

A feature index is randomly selected, and its values are transformed using a logistic function to obtain masking probabilities. These probabilities are then used to sample the indicator vector S, which determines which labels are replaced by a placeholder.

A.3 MAR2

In MAR2, missingness depends on all features:

$$P(S = 1 \mid X, Y) = P(S = 1 \mid X).$$

A score is computed as a normalized sum of all feature values and transformed via a logistic function to obtain masking probabilities. The indicator vector S is then sampled accordingly, and missing labels are introduced in the same way as before.

A.4 MNAR

In MNAR (Missing Not At Random), missingness depends on both X and Y:

$$P(S = 1 \mid X, Y).$$

A combined score based on feature values and the target variable is computed and passed through a logistic function to obtain masking probabilities. The indicator vector S is sampled from these probabilities, and labels corresponding to $S = 1$ are replaced with a placeholder.

B Preparing Datasets

To evaluate the models, four datasets of varying dimensionalities, sizes, and intrinsic imbalances were utilized.

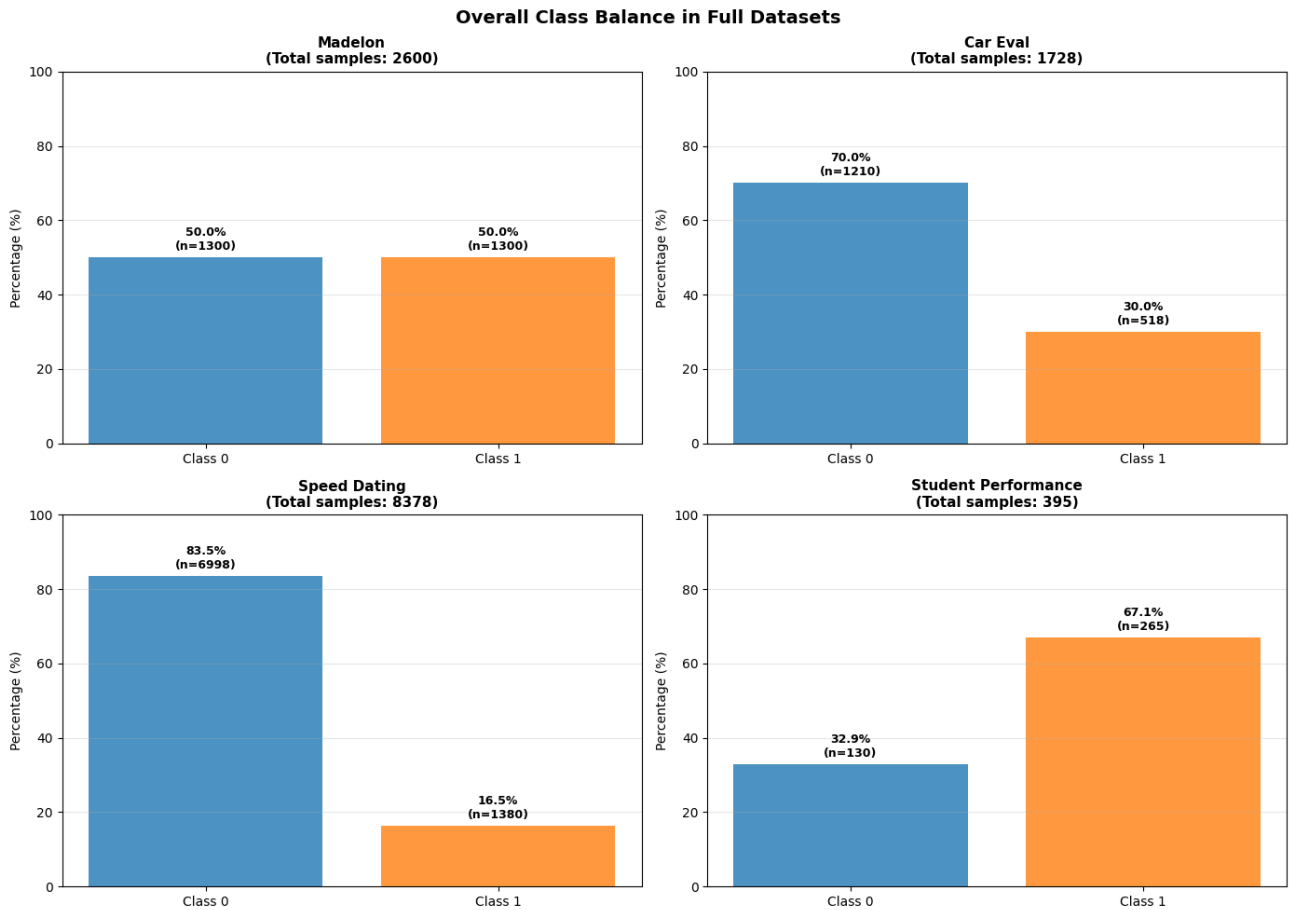
1. **Car Evaluation Dataset** consists of 1728 instances described by 15 features. The task is to classify whether a car is acceptable or not based on attributes such as *number of doors*, *luggage boot size*, and *safety level*.

2. **Madelon Dataset** is an artificial dataset created for the NIPS 2003 feature selection challenge. It is particularly challenging due to its highly multivariate and non-linear structure. The dataset comprises of 2600 instances with 500 features, many of which are redundant or irrelevant.

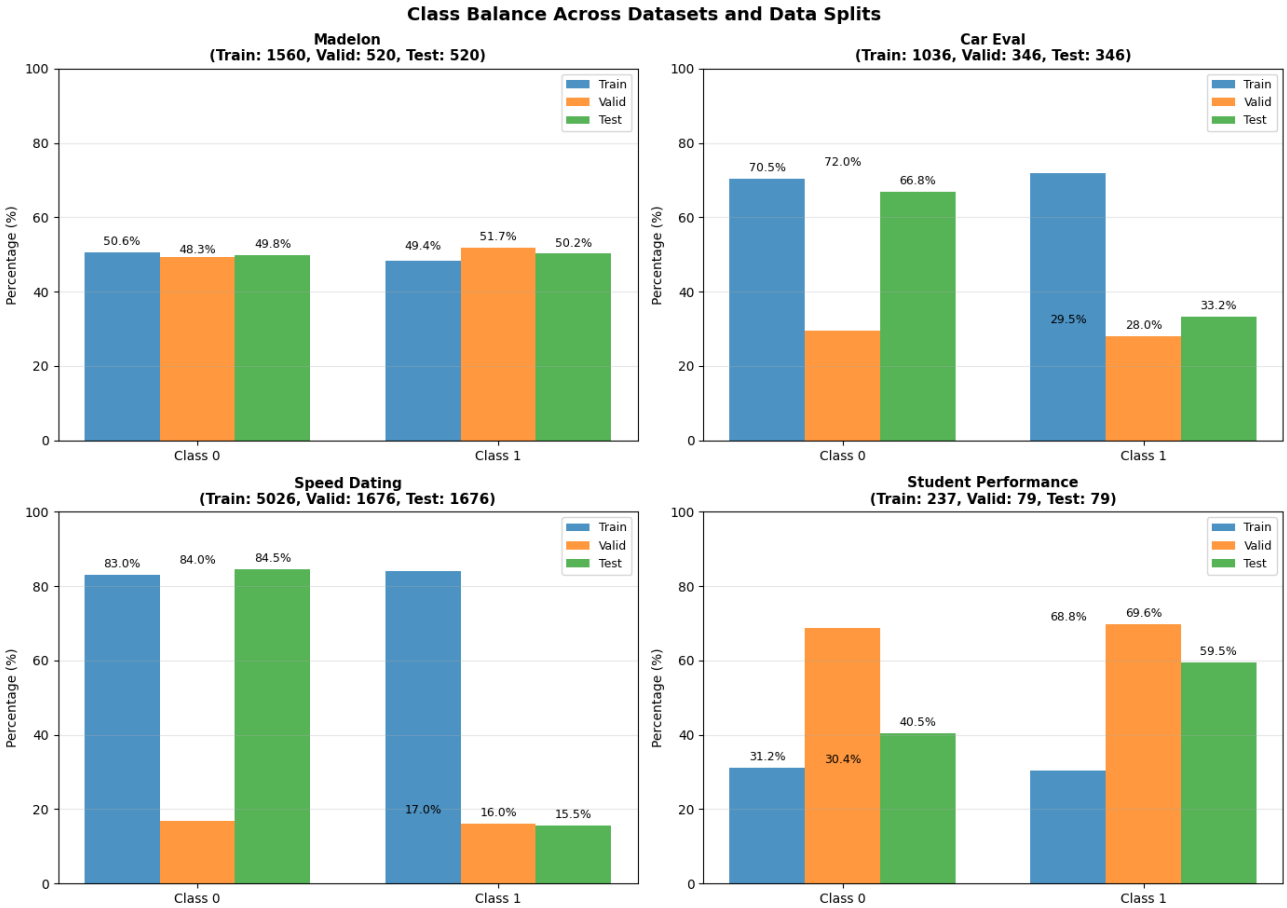
3. **Student Performance Dataset** aims to predict whether a student passes a mathematics course. It includes academic results along with demographic, social, and school-related attributes. The dataset contains 395 instances described by 26 features.

4. **Speed Dating Dataset** addresses a binary classification problem of predicting whether a match occurs between participants. The data was collected during experimental speed dating events and includes features describing participants' preferences, attributes, and interactions. It consists of 8378 instances and 329 features.

To provide a clear view of the data landscape, the overall sample sizes and the detailed class distribution across our training, validation, and test splits are visualized in Figure 4.



(a) Overall sample count comparison across four datasets.



(b) Detailed class balance (0 vs 1) breakdown for Training, Validation, and Test splits.

Figure 4: Visualization of dataset sizes and class distributions.